

Claude Code for research

Bahlo lab meeting

Stuart Lee

2026-05-12

**Claude Code is
research infrastructure,
not autocomplete.**

Three threads today

1. How it works.
2. How I use it.
3. What to watch for.

Then group activities.

How it works

An LLM chat is one long prompt that grows with every turn

```
<|start|>system<|message|>You are ChatGPT, a large language model trained by OpenAI.  
Knowledge cutoff: 2024-06  
Current date: 2025-06-28  
Reasoning: high  
# Valid channels: analysis, commentary, final.  
Channel must be included for every message.  
Calls to these tools must go to the commentary channel: 'functions'.<|end|>  
  
<|start|>developer<|message|># Instructions  
Be concise and technical. Provide code examples when relevant.  
  
# Tools  
## functions  
namespace functions {  
  type web_search = {<|:  
    query: string,  
    num_results?: number,  
  } => any;  
} // namespace functions<|end|>  
  
<|start|>user<|message|>What's new in the latest version of polars?<|end|>  
  
<|start|>assistant<|channel|>analysis<|message|>User asking about latest polars version.  
My knowledge cutoff is 2024-06, need to search for recent updates.<|end|>  
  
<|start|>assistant<|channel|>commentary<|message|><|call|>functions.web_search  
{  
  "query": "polars python latest version new features", "  
  "num_results": 5  
><|end|>  
  
<|start|>tool<|message|>[{"title": "Polars 1.8.0 Release Notes", "snippet": "New  
streaming engine, improved join performance, added pivot_wider...", "url": "..."}]<|end|>  
  
<|start|>assistant<|channel|>final<|message|>Polars 1.8.0 (latest as of Nov 2025)  
includes:  
  
- **Streaming engine overhaul**: Better memory efficiency for large datasets  
- **Join performance**: 2-3x faster for certain join types  
- **New API**: 'pivot_wider()' for reshaping data  
- **LazyFrame improvements**: Enhanced query optimization  
  
Check the release notes for migration details if upgrading from older versions.<|return|>
```

system prompt
(written by openai)

developer
message
(tool definitions)

user
prompt

thinking

tool call

tool
results

final response

ANATOMY OF A
MODERN LLM
CHAT

You
typically
see only
these
two

A token is a subword piece — about three-quarters of a word, four characters on average. The context window is measured in tokens (Sonnet 4.6: 200K standard, 1M premium).

Confident wrong answers are a feature of the architecture

Not looking up facts. Generating fluent text.

Fluent and **wrong** sound the same.

The context window is the model's only memory

Context window (this turn)

- file you opened
- error you pasted
- tool result

= the model's only memory

Not in the buffer:

previous session

files you didn't open

tool calls in other turns

If it “forgot” — it never had it.

Training data has a freshness date



Grounding is what turns guessing into reasoning

Show the actual file, the actual error, the actual data.

What's in the buffer matters more than **which model** you chose.

Web chat to agentic

In a web chat, you are the bridge between model and reality

You read code in. You copy answers out. You re-paste errors.

The model has no hands. You are the hands.

Fine for one-off questions. Does not scale to a workflow.

An agent reads files, runs commands, loops on results

1. **Think**
2. **Act** — read, run, call.
3. **Observe**
4. Repeat.

Can do unattended

Read files, run shells, edit, web, MCP.

Cannot do

Know your goal unless you say it.

More autonomy = more chances to be **wrong** unnoticed.

MCP servers turn Claude into a research client

First-party, free:

- **PubMed** — search, metadata, full text.
- **bioRxiv / medRxiv** — preprints by topic, author, funder.
- **Google Drive, Gmail, Calendar** — notes, drafts, scheduling.

The lever that makes Claude useful for **research**, not just code.

How I actually use it

TDD keeps the agent honest on the C9orf72 pipeline

1. Plan mode — no code yet.
2. Plan: target function, its tests, the simulation.
3. Approve. Failing tests, then function, then green.
4. Read every diff.

Plan first, execute second — every iteration

Plan

- Function?
- Inputs, outputs?
- Property preserved?
- Simulation that proves it?

Execute

- Tests fail.
- Tests pass.
- Read the diff.
- Commit.

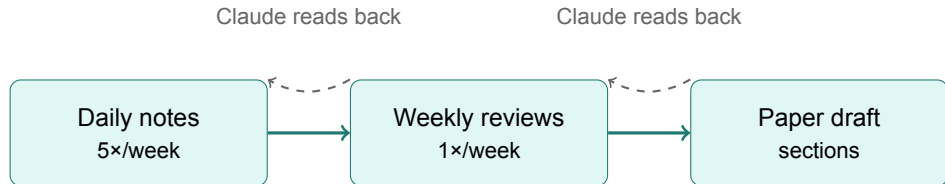
A bug in a web app frustrates a user. A bug in my analysis ends up in a **paper**.

/capture, /meeting, /review make Claude a lab-notebook collaborator

- /capture — thought, paper, fragment → structured note.
- /meeting — Zoom transcript → actions, decisions, follow-ups.
- /review — week's end → summary, what's stuck, next week.

Not the author. The structure that lets me **write more** of my own thinking down.

Daily notes feed weekly reviews feed paper drafts



Methods come from March's notes. Discussion comes from when it first surprised me.

Context management

Context is the bottleneck, not the model

CODING SESSION
WITH AI



next few slides based
heavily on HumanKeyar

Context Engineering
is the new
Prompt Engineering

Near the limit, performance
degrades.

What you put in beats how you
phrase it.

As you approach the
context window,
performance worsens

... remember
performance $\sim$ correctness $\cdot$ completeness
size

CLAUDE.md and skills give Claude durable memory

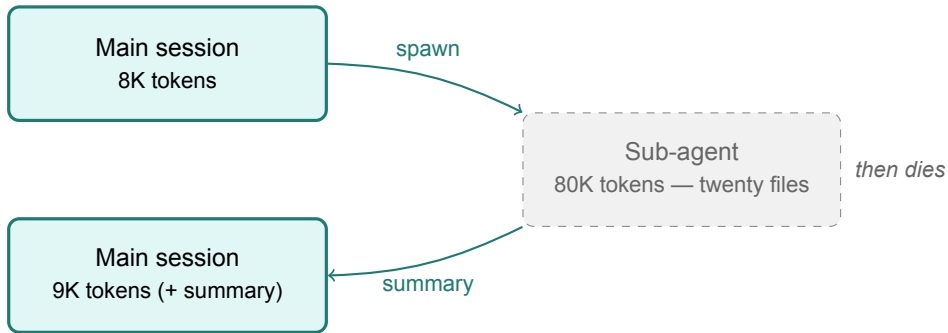
CLAUDE.md — always loaded.

- Conventions
- Domain vocabulary
- Project anti-patterns
- Pointers to source-of-truth files

Skills / slash commands — on demand.

- Reviewing a PR
- Bumping a package version
- Drafting a session log

Sub-agents read; design docs persist; restart before drift



Design docs survive sessions. Repeating corrected mistakes = drift — restart, reload.

Verification

Git is Claude's other memory — and your safety net

- **Rollback is cheap.** `git reset` undoes nonsense.
- **Memory is auditable.** "What changed in the last three commits?" — concrete, not guessed.

Commit early, commit often.

Read the diff, every time

- Code → run the test, quote the output.
- Data → row counts, dtypes, NA rates, before / after.
- Plot → render it. Look at it.
- Model → predicted vs. actual.

“It should work” is **not** verification.

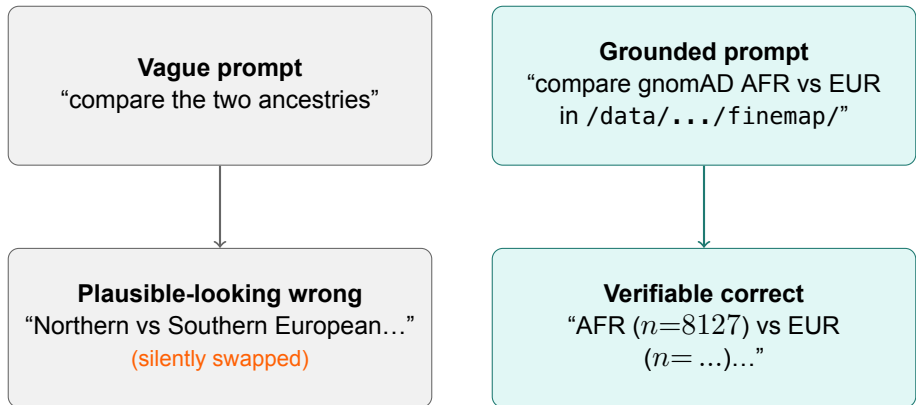
Simulate with a known truth; check it's recovered

For any step that feeds a figure or a table:

1. Generate data with known ground truth.
2. Run the pipeline.
3. Confirm it recovers the truth within tolerance.

Substitute for cross-software replication. The simulation is the test fixture.

When it's confused, re-ground — don't argue



Pitfalls

Six ways this goes wrong

Premature action

plan first

Confabulated paths

verify by reading

Sycophancy

it will agree if you push

Cost runaway

set a budget

Over-reliance

risky for students mid-training

Vibe coding

runs, but no one understands it

Lab policies

Data egress is the binding constraint, not SSH topology

Fine

- Code-only HPC repos
- Pipeline + simulation code
- Manuscript drafts (code-only dirs)

Not fine without policy

- UKB, GEL, clinical data
- Mixed code/data directories
- "It's on the HPC" as control

Not *where* the agent runs. *What bytes leave the building.*

Ask IT for an Enterprise tier and a deny-list

Concrete ask

Enterprise tier (zero data retention) + a documented **deny-list** for controlled-access directories.

Until then: laptops ✓, code-only HPC ✓, controlled data **off-limits**.

Where to start

You all have access — try this Monday

Three things this week

1. Open Claude Code in a real project. Propose a `CLAUDE.md`. Commit.
2. One recurring task → slash command.
3. Pick an activity from `ACTIVITIES.md`.

Claude Code is research infrastructure, not autocomplete.